

Agentic Workflow — Complete Reference

2026-07-10 (last updated 2026-07-11, session 3) · Polished periodic snapshot — day-to-day edits now happen in agentic-workflow.md, regenerated here only after a batch of real changes. Replaces agentic-workflow-setup.pdf, agentic-workflow-cheatsheet.pdf, and agentic-workflow-session-update.pdf

Current state: fully built and verified working. WezTerm + herdr + Neovim + Claude Code + Codex + OpenCode, all wired to local (omlx) and cloud (Claude) inference, voice input via Parakeet v3, the full axi tool family, and Kun's productivity stack (lavish, treehouse, no-mistakes, gnhf, firstmate) are all installed, declared in Nix, and tested end-to-end.

1. Quick Reference — Keys & Commands

The one mental model: a prefix key (herdr = Ctrl+b) precedes multiplexer commands. In Neovim, a leader key (Space) precedes custom commands. Press prefix/leader, release, then the next key.

herdr — prefix Ctrl+b

Ctrl+b "	split pane horizontal
Ctrl+b %	split pane vertical
Ctrl+b h/j/k/l	move focus
Ctrl+b c	new tab (fresh shell)
Ctrl+b &	close tab
Ctrl+b w	workspace picker
Ctrl+b g	go to (jump)
Ctrl+b y	copy mode (v/Space select, y/Enter copy)

Sessions persist on a background herdr server — closing the terminal doesn't kill your work. But see §9 for why that same persistence can cause stale-environment bugs.

Neovim — leader Space

h j k l	left/down/up/right
w / b	next/prev word
0 / \$	start/end of line
gg / G	top/bottom of file
Nj / Nk	jump N lines (relative #s in gutter)
i / a / o / O	insert modes
Esc	leave insert + saves (custom map)
dd / yy / p	delete / copy / paste line
u / Ctrl+r	undo / redo

Agents

cc	alias: claude --dangerously-skip-permissions (cloud)
co	alias: codex --sandbox workspace-write --ask-for-approval never (routed to local omlx, default model Qwen3.6-35B-A3B-8bit as of 2026-07-11). Replaces the old --full-auto flag, removed in Codex 0.128.
oc	alias: omlx launch opencode --model mlx-community--Qwen3.6-35B-A3B-8bit (routed to local omlx). Prefer this over co for context-heavy work — see §5.
/	slash commands (skills, config, /lavish, /no-mistakes)
plan mode	ask the agent to plan before building
Esc	interrupt agent mid-task

Voice input (Parakeet v3)

hold ˆ Option Right	push-to-talk: hold, speak, release
ˆ Option + `	toggle: press to start, press to stop

Global hotkey — works in any focused app (herdr pane, Messages, anywhere), agent-agnostic. No vocab-priming with Parakeet (Whisper-only feature) — see §6.

Git essentials

git status	what changed
add	alias: git add .

Space f	find files	git commit -m	commit staged
Space s	search/grep project	push / pull	aliases for git push/pull
Space b	switch buffers	m	alias: git switch main
Space e	file browser (Oil)	lazygit	full-screen git TUI
Space g	git UI (Neogit)		
gd	go to definition		

Protocol: HTTPS via gh auth login, not SSH — background daemons (no-mistakes, gnhf, firstmate crew) can't reliably reach an SSH agent socket, but the gh-issued token in Keychain works for any process.

omlx (local inference)

omlx status	server/client health
omlx restart	restart + rescan for new models
omlx launch <tool> -- model <id>	wire a tool to omlx
hf download <repo>	pull a new model (omlx's bundled CLI)

2. Component Status

COMPONENT	STATUS	NOTES
WezTerm, herdr, Neovim, Claude Code, node/npx	done	core ship, unchanged since original setup
lavish	done	project + global skill; visual planning
skill-creator	done	Anthropic's own — lets agent author new skills
gh-axi	done	GitHub ops, ~3x cheaper/2x faster than GitHub MCP
chrome-devtools-axi	done	browser automation, ~40% fewer tokens than raw MCP
quota-axi	done	usage tracking across Claude/Codex/Cursor/Copilot/Grok
treehouse	done	pooled git worktrees, ~/.local/bin/treehouse
no-mistakes	done	validation pipeline, daemon running
gnhf	done	via npx gnhf (no global install — see §9)
firstmate	done	cloned to ~/github/kunchenguid/firstmate
gh (GitHub CLI)	done	authenticated, HTTPS protocol
omlx	done	v0.5.0, serving mlx-community/Qwen3.6-35B-A3B-8bit (switched from 27B dense 2026-07-11, ~5.6x faster). GPU memory ceiling and context-window policy hardened 2026-07-10 — see §5
Codex CLI → omlx	done	wired to omlx, verified via codex exec. Bundled model catalog doesn't recognize local models — cosmetic metadata warning only, see §5

OpenCode (oc) → omlx	done	wired via <code>omlx launch opencode</code> ; preferred over Codex for context-heavy sessions since omlx writes real metadata into its config — see §5
Claude Code → omlx	decided	deliberately cloud-only — not wired, by choice
OpenSuperWhisper + Parakeet v3	done	installed, permissions granted, verified pasting text
Kokoro TTS (voice output)	not built	bundled in omlx, unused — see §8

3. Memory Files & Skills — How the Agent Learns Your Preferences

Global memory (`~/ .claude/CLAUDE.md`, symlinked to `AGENTS.md` for Codex/OpenCode too) — personal preferences, loaded into every session. Keep it short (~27 lines); it costs tokens on every request. Yours already has: never use em dash, don't weight development cost in technical decisions, reproduce bugs end-to-end before fixing, be pixel-perfect on UI, fix lint/test issues you notice even if unrelated to the task.

Project memory (`CLAUDE.md` / `AGENTS.md` in a repo root) — grows by correcting the agent and telling it to "remember." This is the collective learning of every session in that project.

Skills — conditionally-loaded procedures (only read when relevant), installed via `npx skills add <owner/repo> --skill <name> [-g]`. Move verbose/conditional instructions out of memory files and into skills to keep the memory file lean.

Skill security: a skill can run anything on your machine — API key exfiltration, arbitrary shell access. Do not install skills from unverified sources, even popular ones (a benchmark cited in Kun's video showed a 177k-star skill making agent output measurably worse while using 5% more tokens). Only the official kunchenguid axi catalog (`axi.md`) and Anthropic's own skills were installed here. Scanner "Med Risk" ratings on `gh-axi` / `chrome-devtools-axi` / `quota-axi` reflect that they can act on GitHub/a browser/your usage data — that's the capability working as intended, not a red flag on the code itself.

4. The Workflow Loop

1. Plan	Discuss in lavish (interactive HTML artifact, browser-based annotation) — decide requirements here, not by reading walls of text
2. Build	Agent works the middle — step away, start other tasks
3. Validate	no-mistakes: branch → adversarial review → e2e test (with evidence) → docs → lint → PR, babysat until merged
4. Judge	Review the PR, weighted by its risk assessment — not every diff
5. Scale	treehouse (parallel worktrees, no manual cleanup) + firstmate (manages the crew so you talk to one agent, not many)

Captain's mindset: you're an engineering director, not a diff-reviewer. Set good process, trust the pipeline, spend your energy on what to build — competition, users, direction — not line-by-line review.

Daily muscle-memory drills

- Split a herdr pane, run an agent in one, nvim in another
- Jump with relative line numbers (`Nj` / `Nk`) instead of scrolling
- Space f a file, Space s a symbol — no mouse, ever

- Dictate instead of type whenever the prompt isn't a URL/path
- Detach herdr, reattach — the session is still there

Goal: hands never leave the keyboard. That's the whole point — staying in flow.

5. Local Inference — omlx

omlx is an MLX inference server purpose-built for coding agents on Apple Silicon: continuous batching, paged SSD KV-caching (repeated large contexts go from 30–90s time-to-first-token down to 1–3s), and native OpenAI + Anthropic API compatibility — meaning it's a drop-in local backend, not just a generic chat server. Verified legitimate via a real discussion thread in Apple's own ml-explore/mlx GitHub repo.

Current setup

- Installed via private Homebrew tap `jundot/omlx`, trusted and declared in `configuration.nix`
- Running as a background service (`homebrew.mxcl.omlx launchd` job), auto-starts at login
- Serving `mlx-community/Qwen3.6-35B-A3B-8bit` as of 2026-07-11 (MoE, 3B active params) — switched from the prior default, `mlx-community/Qwen3.6-27B-8bit (dense)`. This machine's own memory-safety cap enforces a hard 65,536-token ceiling regardless of either model's native window — see "GPU memory & context-window hardening."
- Codex CLI is wired to it (`~/codex/config.toml`, provider `omlx`, base_url `http://127.0.0.1:8000/v1`) — verified working end-to-end via `codex exec`
- OpenCode is also wired to it via `omlx launch opencode --model <id>` (the `oc` alias) — preferred over Codex for context-heavy work, since omlx writes its real, live context-window metadata into OpenCode's config at launch time instead of Codex guessing from a static bundled catalog
- Claude Code stays cloud-only — a deliberate choice, not a gap. Claude's cloud models remain the higher ceiling for this agent's actual coding work.

Trust lesson from this setup: a model already on disk (...Claude-4.6-Opus-Deckard-Heretic-Uncensored-Thinking-8bit) was deleted before use. "Heretic" is a known automated ablation technique that strips safety/refusal training — it measurably degrades coherence and instruction-following even when done carefully, and there's no legitimate coding/planning task that benefits from an uncensored model. The name also falsely implied lineage with Claude Opus. Before running any downloaded model: check the name for uncensored/ablated branding and fabricated lineage claims to closed frontier models. Prefer official `mlx-community` releases.

Model swap: Qwen3.6-27B-8bit → Qwen3.6-35B-A3B-8bit (2026-07-11)

Motivation: the dense 27B model was clearing ~16.8 tok/s through Codex, well under a ~30 tok/s target for a smooth interactive loop.

Considered and set aside: `antirez/ds4` ("DwarfStar"), a dedicated engine for DeepSeek V4 Flash (284B total/13B active) — real published 34.27/25.9 tok/s (short/11.7K-context) on this exact M5 Max 128GB class, but beta-quality, a separate server from omlx, alpha-quality agent glue. GLM-5.2 and MiniMax-M3 top the open-weight intelligence leaderboards but are physically too large for 128GB (753B and 427B total params).

Adopted: `mlx-community/Qwen3.6-35B-A3B-8bit` — same family omlx already has custom kernels for, MoE with ~3B active params/token, zero new integration (just `hf download + omlx restart`).

Measured on this machine: raw `mlx_lm.benchmark` ~104 tok/s short-prompt / ~94 tok/s at 11.7K context (vs ~16.8 tok/s baseline — MoE barely degrades under long context, unlike the dense model); ~88 tok/s through the live omlx server. Real `codex exec` run, identical prompt on both models: 42.3s wall-clock (27B, cache warm) vs 14.3s (35B-A3B, cache cold — a harder case for it) while producing 53% *more* output tokens. **Net: ~5.6x faster generation throughput, ~2.9x faster wall-clock on a real task.**

BENCHMARK	QWEN3.6-27B (OLD)	QWEN3.6-35B-A3B (NEW)	GAP
SWE-bench Verified	77.2	73.4	-3.8
SWE-bench Pro	53.5	49.5	-4.0
Terminal-Bench 2.0	59.3	51.5	-7.8
LiveCodeBench v6	83.9	80.4	-3.5
MMLU-Pro	86.2	85.2	-1.0
GPQA Diamond	87.8	86.0	-1.8
AIME26	94.1	92.7	-1.4

Dense wins every benchmark (Qwen's own published numbers), mostly by 1-4 points; Terminal-Bench (closest proxy to real agentic CLI tool-use) shows the largest gap at ~8 points. Judged worth it for the ~3-5.6x speed gain. The 27B weights stay on disk, not deleted — for the rare max-quality task: `omlx launch codex --model mlx-community--Qwen3.6-27B-8bit`.

GPU memory & context-window hardening (set 2026-07-10)

omlx hard-crashed once under memory pressure. Root cause: macOS resets the `iogpu.wired_limit_mb` sysctl (the GPU-usable share of unified memory) to a low default on every reboot, undercutting omlx's own internal ~122GB target — so instead of gracefully rejecting an oversized request, it ran out of wired memory and hard-panicked. Fixed with two changes, both durable across reboots:

- `iogpu.wired_limit_mb=124928` (122GB, matching omlx's internal target exactly), applied via a `launchd.daemons` entry in `configuration.nix` with `RunAtLoad = true` — not a plain `system.activationScripts` entry, which has a known nix-darwin issue where it doesn't reliably re-run from `org.nixos.activate-system` after a reboot (only on `darwin-rebuild switch`)
- `max_context_window_policy: 65536` in `~/omlx/settings.json` (under `sampling`) — caps requests at the server level so an oversized context is rejected up front instead of exhausting memory via unbounded KV-cache growth

Codex's fabricated context-window metadata (investigated, not fixed — by design)

Codex's TUI reports a "Model metadata... not found" warning and internally uses `model_context_window: 258400` for this model — nowhere close to the real 65,536-token cap omlx enforces. This is **not** an omlx bug: omlx's own `/v1/models` endpoint correctly reports `max_model_len: 65536`. Codex's bundled model catalog simply doesn't recognize custom/local models, so it substitutes a generic, provider-agnostic fallback constant ($258400 = 272000 \times 0.95$) — confirmed by an upstream GitHub issue showing the identical number for GPT-5.5, an unrelated official OpenAI model on a different provider entirely.

Deliberately **not** patched via Codex's `model_context_window` config override: a confirmed upstream bug (openai/codex #16068) means manually setting that value can permanently break auto-compaction after the first context overflow, recreating a crash-loop-class problem. The safe mitigations instead: prefer `oc` for anything context-heavy (already accurate, see above), and for long `co` sessions run `/compact` manually rather than trusting Codex's automatic trigger, since a user-initiated compaction doesn't go through the buggy overflow-handling path.

Adding another local model later

```
/opt/homebrew/Cellar/omlx/<version>/libexec/bin/hf download mlx-community/<repo>
omlx restart # required – server doesn't hot-reload; this triggers a rescan
```

6. Voice Input — OpenSuperWhisper + Parakeet v3

Local, free, open-source dictation. Installed as a cask, declared in `configuration.nix`.

MODEL	WHY
Whisper V3 Large / Medium / Small	not used — slower, and hallucinates during silence (autoregressive decoder always tries to produce text)
Parakeet v2	marginally more accurate on English only (6.05% vs 6.32% WER), no multilingual
Parakeet v3 — chosen	beats Whisper Large V3 on accuracy (6.32% vs 7.44% WER) at ~23x the speed (~3,333x real-time factor vs ~146x), zero silence hallucination, adds 25-language support at negligible cost

Known limitation: the vocabulary/hotword priming feature (feeding it jargon like "herdr", "omlx" so it transcribes correctly) only works with Whisper models in this app, not Parakeet. Accepted tradeoff — fall back to typing or manual correction on jargon-heavy prompts, same as Kun's own advice to type URLs/paths rather than speak them.

7. Not Yet Built — Kokoro TTS (Voice Output)

What it is: omlx already bundles `mlx-audio`, which includes the Kokoro text-to-speech model — the same local server that's serving your coding model can also generate spoken audio. This is genuinely on your machine right now, just unused.

What's missing: neither Claude Code nor Codex has a built-in "read the response aloud" feature. Nothing currently pipes either agent's text output through omlx's TTS endpoint.

What building it would take: a small hook/script that (1) captures an agent's final response text, (2) POSTs it to omlx's TTS endpoint, (3) plays the resulting audio. Candidate integration points: a Claude Code Stop hook (fires when the agent finishes responding), or a wrapper skill. Not started — flagged here so it isn't forgotten, not because it's difficult.

8. Why This Is a Different Way of Working

The individual pieces (an agent CLI, a terminal multiplexer, git worktrees) aren't new. What's different here is that every point of friction in the loop has a purpose-built fix, and they compose:

- **Typing is the bottleneck → removed.** Voice input at ~3x typing speed, global and agent-agnostic, means prompting an agent costs almost nothing. You stop rationing how often you talk to it.
- **Reading walls of text is the bottleneck → removed.** lavish turns a plan into an annotatable visual artifact. You point at what's wrong instead of writing a paragraph explaining it.
- **Reviewing every diff is the bottleneck → removed.** no-mistakes' pipeline catches what a human reviewer would catch, with recorded evidence, and escalates only genuine judgment calls. You spend review time proportional to risk, not to line count.
- **One agent at a time is the bottleneck → removed.** treehouse makes parallel worktrees free (no manual cleanup, no "which branch was I on" tax), so running 3 agents costs the same attention as running 1.
- **Managing multiple agent sessions is the bottleneck → removed.** firstmate is a single point of contact that distributes work across a crew and reports back with finished PRs — you stop being the one doing the context-switching.
- **Tool inefficiency is the bottleneck → removed.** gh-axi/chrome-devtools-axi cut token cost and latency on the two things agents do constantly (GitHub, browser), which compounds across every task, every day.

- **Cost/availability is the bottleneck → removed (optionally).** omlx gives Codex and OpenCode a zero-marginal-cost, fully local, fully private backend for iteration, with Claude Code still available for the highest-stakes work.

None of this replaces judgment — it moves your job from producing and checking individual changes to directing a process that produces and checks itself, which is the actual leverage.

What a real day looks like

Morning: voice-driven planning

Open WezTerm (auto-attaches to herdr). Hold `⌘` Option: "I'd like to consolidate the two dashboard buttons since their functionality overlaps, and make it feel more like an achievement system — give me some options." lavish auto-activates (it's a skill, not a command you type), opens a browser artifact with the options laid out visually instead of a wall of text. Click to annotate, decide, say "start building." Switch panes.

Midday: parallel scaling

`Ctrl+b c` for a new tab, treehouse for a fresh worktree, `cc`, dictate the next task. Repeat twice more for two unrelated fixes. Three agents now working in isolated worktrees, each set to run no-mistakes on completion. You're not touching any of them until the status bar says done.

Afternoon: judgment, not review

A PR shows up flagged low-risk with recorded e2e evidence (screenshot/video/log) — skim the summary, merge. Another is flagged higher-risk because it touched auth — actually read that diff. Time spent is proportional to risk, not to how much code changed.

Overnight: autonomous loop

`npx gnhf "reduce flakiness in the test suite until none remain" --max-tokens <cap>` in a clean worktree before bed. Wake up to a branch of small, individually-committed, tested changes — or nothing, if it never found a problem worth fixing. Either way, zero of your time spent overnight.

Anytime: cost-free iteration

`co` for quick, throwaway exploration, or `oc` for anything context-heavy (see §5) — both routed through local omlx, zero API cost, fully private, no rate limits. Reach for `cc` when the stakes or complexity actually call for Claude's ceiling.

9. Maintenance, Upgrading & Future-Proofing

Golden rule for this machine: everything durable is declared in `~/github/JalenMickey/dotfiles/configuration.nix` (Homebrew taps/brews/casks) or `home.nix` (nixpkgs packages, env vars, session PATH). `homebrew.onActivation.cleanup = "zap"` is intentional and will silently remove anything installed ad-hoc on the next `./rebuild.sh`. If you `brew install` something directly to test it, declare it before your next rebuild or expect it gone.

Never run `brew install/reinstall/upgrade` at the same time as `./rebuild.sh`. Both touch the Homebrew Cellar and will lock-collide (happened exactly once, mid-session, with omlx). Wait for one to finish before starting the other.

Known gotchas (all hit and resolved across setup sessions)

SYMPTOM	REAL CAUSE	FIX
New <code>home.nix</code> env var doesn't show up anywhere, even a fresh terminal	the herdr server is a long-running background process (survives WezTerm quitting); new panes inherit its stale environment, not a fresh one	<code>herdr server stop</code> , then relaunch herdr — new panes inherit a genuinely fresh environment only after the server itself restarts

App records/detects fine but silently does nothing (e.g. dictation doesn't paste anywhere, not even other apps)	macOS Accessibility/TCC permission grants often don't apply to an already-running process — it cached its trust state at launch, before you flipped the toggle	fully quit and relaunch the app after granting permission, not just toggle the setting
omlx broken after a rebuild (bad interpreter path)	an undeclared transitive dependency (python@3.11) got zap-removed as an "orphan"	declare every real dependency explicitly in <code>configuration.nix</code> , not just the top-level package (see <code>rust</code> , kept for omlx's from-source rebuilds)
global <code>npm install -g</code> fails outright	this machine's npm prefix is a read-only Nix store path	use <code>npx <pkg></code> for on-demand invocation instead of a global install (works for <code>gnhf</code> , <code>lavish</code> , all <code>axi</code> tools)
omlx hard-crashed under memory pressure	<code>iogpu.wired_limit_mb</code> resets to a low default on every reboot, undercutting omlx's own ~122GB internal target, so it panics instead of gracefully rejecting oversized requests	<code>iogpu.wired_limit_mb=124928</code> via a <code>launchd.daemons</code> entry (<code>RunAtLoad</code>) in <code>configuration.nix</code> , plus <code>max_context_window_policy: 65536</code> in <code>~/.omlx/settings.json</code> — see §5
co breaks with error: unexpected argument '--full-auto' found	Codex 0.128 removed the <code>--full-auto</code> flag	use <code>--sandbox workspace-write --ask-for-approval never</code> (now the <code>co</code> alias) — not <code>--dangerously-bypass-approvals-and-sandbox</code> , which removes the sandbox entirely rather than just skipping approval prompts
Codex reports fabricated context-window metadata (258400) for the local model, unrelated to the real 65536 cap	Codex's bundled model catalog doesn't recognize custom/local models and falls back to a generic, provider-agnostic constant (272000 × 0.95) — see §5	no safe config fix (upstream bug #16068 can permanently break auto-compaction). Use <code>oc</code> for context-heavy work; run <code>/compact</code> manually in long <code>co</code> sessions

Update commands (verified)

Skills (lavish, axi family, skill-creator)	<code>npx skills update -g</code>
Codex CLI	<code>codex update</code>
OpenCode	<code>brew upgrade opencode</code> (declared in <code>configuration.nix</code> , so <code>./rebuild.sh</code> also keeps it current)
herdr	<code>herdr update</code> (or <code>herdr channel set stable preview</code> to change release channel)
firstmate	<code>/updatefirstmate</code>
gnhf	nothing to do — <code>npx gnhf</code> always resolves latest published version
treehouse / no-mistakes	re-run the original install script (idempotent, always fetches the latest release)
omlx	<code>brew upgrade jundot/omlx/omlx</code>
— recompiles from source every time (~3 min, no prebuilt bottle on this private tap); never run this at the same time as <code>./rebuild.sh</code>	
gh, node, and everything else in <code>home.packages</code>	bump the flake input (<code>nix flake update</code> in the dotfiles repo) then <code>./rebuild.sh</code>

Periodic health checks

- `gh auth status` — tokens can expire
- `omlx diagnose menubar` — built-in self-diagnostic for install/runtime issues

- Check disk usage under `~/ .omlx/models` and `~/ .cache/huggingface` — local models are 20–30GB+ each and accumulate
- Before downloading any new local model: re-run the same name/lineage sanity check from §5 (uncensored/abliterated branding, fabricated lineage to closed models)
- Re-run `./rebuild.sh` occasionally even with no known changes — catches config drift early, cheaply

Secrets hygiene

The dotfiles repo is public. Pattern used for `OMLX_API_KEY` — reusable for any future credential:

```
home/<name>.local          # plaintext value, gitignored (home/*.local)
home.nix reads it via:
  lib.optionalString (builtins.pathExists file) (lib.strings.trim (builtins.readFile file))
```

This keeps the *shape* of the config declarative and public while the actual secret value never enters git history.